

Python Cheatsheet & Quick Reference Guide

1. Basic Syntax & Data Types

Data Types

```
# Numbers
x = 10      # int
y = 3.14    # float
z = 1 + 2j   # complex

# Strings
s = "Hello"
multiline = """Line 1
Line 2"""

# Booleans
is_true = True
is_false = False

# None
empty_val = None
```

Data Structures

```
# Lists (Mutable, ordered)
lst = [1, 2, 3]
lst.append(4)
lst[0] = 0

# Tuples (Immutable, ordered)
tup = (1, 2, 3)

# Dictionaries (Key-value pairs, unordered)
dct = {'a': 1, 'b': 2}
val = dct.get('a', 0) # Returns 1 (or 0 if not found)

# Sets (Unique elements, unordered)
st = {1, 2, 3}
st.add(4)
```

Control Flow

```
# If / Elif / Else
if x > 10:
    print("Large")
elif x == 10:
    print("Ten")
```

```

else:
    print("Small")

# For Loops
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i, val in enumerate(lst):
    print(i, val)      # Prints index and value

# While Loops
while x > 0:
    x -= 1

```

Comprehensions

```

# List Comprehension
squares = [x**2 for x in range(10) if x % 2 == 0]

# Dictionary Comprehension
sq_dict = {x: x**2 for x in range(5)}

```

2. Commonly Used Built-in Functions

- `len(obj)` : Returns the length of an object.
- `type(obj)` : Returns the type of an object.
- `print(obj)` : Prints to standard output.
- `range(start, stop, step)` : Returns a sequence of numbers.
- `enumerate(iterable)` : Returns an enumerate object (index, value).
- `zip(*iterables)` : Aggregates elements from each of the iterables.
- `map(function, iterable)` : Applies function to every item of iterable.
- `filter(function, iterable)` : Constructs an iterator from elements where function returns True.
- `sum(iterable)` : Sums start and the items of an iterable.
- `min(iterable)` , `max(iterable)` : Returns smallest/largest item.
- `sorted(iterable, key=None, reverse=False)` : Returns a new sorted list.
- `abs(x)` : Absolute value of a number.
- `all(iterable)` : Returns True if all elements are truthy.
- `any(iterable)` : Returns True if any element is truthy.

3. Important Modules

`collections` (Container Datatypes)

```

from collections import Counter, defaultdict, deque, namedtuple

# Counter: dict subclass for counting hashable objects
counts = Counter(['a', 'b', 'a', 'c', 'b', 'a'])

```

```

print(counts['a']) # 3
print(counts.most_common(1)) # [('a', 3)]

# defaultdict: dict subclass that calls a factory function for missing keys
dd = defaultdict(list)
dd['missing_key'].append(1) # No KeyError, creates list and appends

# deque: list-like container with fast appends and pops on either end
d = deque([1, 2, 3])
d.appendleft(0)
d.pop() # Removes 3

# namedtuple: factory function for creating tuple subclasses with named fields
Point = namedtuple('Point', ['x', 'y'])
p = Point(11, y=22)
print(p.x, p.y)

```

itertools (Functions creating iterators for efficient looping)

```

import itertools

# count(start, step)
for i in itertools.count(10, 2):
    if i > 15: break
    print(i) # 10, 12, 14

# cycle(iterable) - repeats indefinitely
# itertools.cycle('ABCD') -> A B C D A B C D ...

# chain(*iterables) - combine multiple iterables
combined = list(itertools.chain([1, 2], ['a', 'b'])) # [1, 2, 'a', 'b']

# combinations(iterable, r)
combos = list(itertools.combinations('ABC', 2))
# [('A', 'B'), ('A', 'C'), ('B', 'C')]

# permutations(iterable, r)
perms = list(itertools.permutations('AB', 2))
# [('A', 'B'), ('B', 'A')]

```

datetime (Basic date and time types)

```

from datetime import datetime, date, timedelta

# Current Date and Time
now = datetime.now()
today = date.today()

# Creating specific dates

```

```
specific_date = datetime(2023, 10, 1, 14, 30, 0)

# Parsing from string
dt = datetime.strptime("2023-10-01", "%Y-%m-%d")

# Formatting to string
dt_str = now.strftime("%Y-%m-%d %H:%M:%S")

# Time differences (timedelta)
tomorrow = today + timedelta(days=1)
two_hours_ago = now - timedelta(hours=2)

# Finding the difference
diff = tomorrow - today
print(diff.days) # 1
```